



STIWK2124 Web Engineering

Assignment 3

PREPARED BY:

GROUP 6

NAME	MATRIC NO
DESMOND YAK TEIK JUN	304294
TAN ZHAO BIN	304666
WONG ZHENG QUAN	303872
TAN XUAN JIE	303956
NURANISYA BINTI RAMLAN	301119

PREPARED TO :

Dr. Munya Saleh Saeed Ba Matraf

TABLE OF CONTENT

Table of Contents

1. INTRODUCTION.....	1
2. DESIGN TRADE-OFFS.....	2
3. FUNCTIONALITY AND USER EXPERIENCE	3
4. PERFORMANCE MEASUREMENT.....	9
5. SECURITY RISK AND MITIGATION.....	11
6. TESTING.....	13
7. NEXT STEPS.....	17
8. CONCLUSION.....	18

1. INTRODUCTION

The Accessible Reading List (ARL) is a web application created as part of assignment three in STIWK2124 Web Engineering, it is a full stack web app built by Angular 19 for the frontend, Spring Boot 3 for the backend and MySQL for the persistent data store.

The ARL was designed as a Reading List Dashboard for users to find and browse books and manage their reading list with an easy-to-use interface. The application is secure by providing user authentication with role-based access to function, along with a database to ensure data integrity and controls users' access to data (database) to manage books. Students will view and be able to search for and manage books they wish to read, while Administrators can create and update and delete records of books in the database.

In summary, the ARL is a secure, accessible and efficient web application that allows users to access reading list information. This report will reflect on any design trade-offs that were made during development, review how well the ARL works and meets the needs of the users, present measurable performance data of the ARL, discuss security vulnerabilities / risks and their mitigation strategies for the ARL, and outline the next steps for improving upon the ARL's current functionality.

2. DESIGN TRADE-OFFS

2.1 Basic Authentication VS JSON Web Tokens (JWT)

The team implemented Basic Authentication for write endpoints instead of using JSON Web Tokens (JWT) because it easily with Spring Security, and the Angular authentication interceptor automatically adds the Authorization header to each POST, PUT, and DELETE request, it requires little client-side management of the token. However, a major drawback of Basic Auth is that the credentials are Base64-encoded and sent via HTTP over the Internet. Without using HTTPS, the credentials could be intercepted, and an attacker could easily decode and use the credentials to gain access to protected resources. For local development purposes, the risk is small. For production use, Basic Authentication can only be used with mandatory HTTPS, or it must migrate to JWT because JWT sends stateless, expiration-based tokens without requiring any transmission of credentials each time.

2.2 Admin VS Student

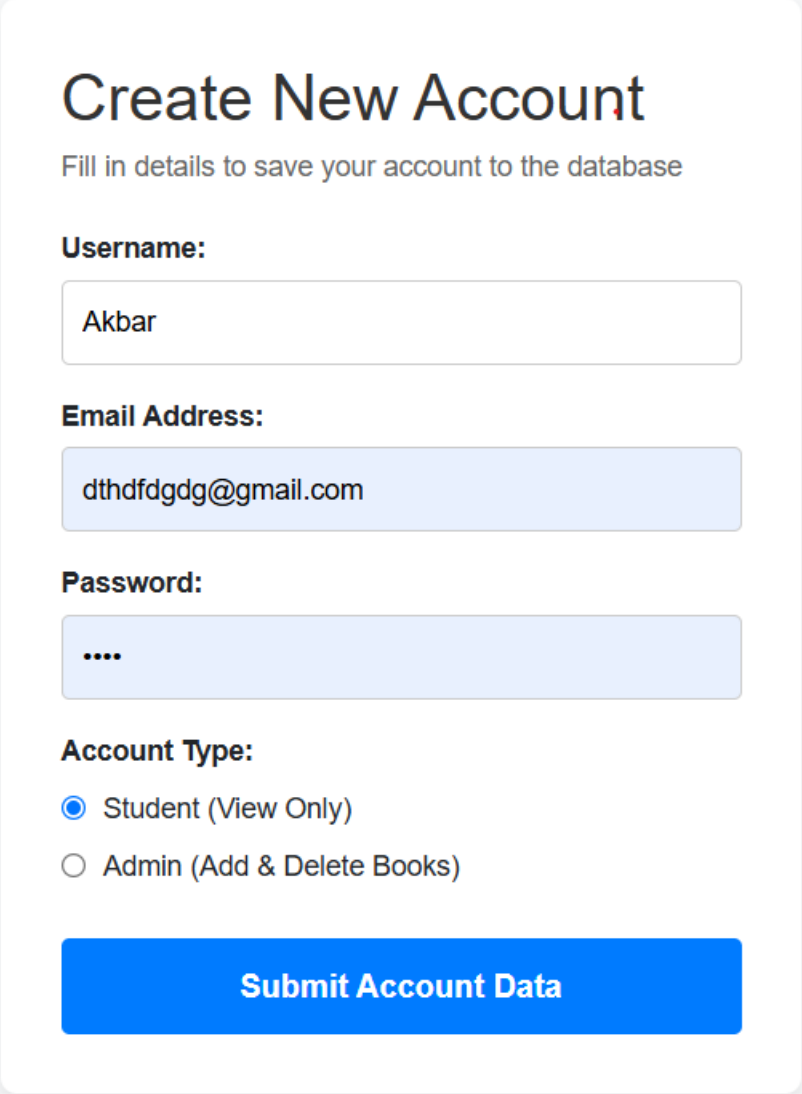
We implemented a two-role system which is administrators who can add books, edit book information, and delete books meanwhile students can only browse, search for, and use the Read Aloud feature. The separation between the two role types is enforced in two ways: on the frontend by conditionally rendering action buttons in the Angular UI based upon the role of the current user. Besides, on the backend with `admin.setRole("ADMIN")` to assign role admin and Spring Security's like `hasAnyAuthority("ADMIN")` rule for write endpoints, this will block POST, PUT, and DELETE requests if the user is not an ADMIN. This approach maintains a simple and appropriately scaled permissions model for this application and does not require over-engineering of a detailed permission model, which would result in needless complexity.

2.3 Deployment Using Docker Compose and Monolithic Architecture

We have a monolithic Spring Boot application by developing in one codebase instead of multiple microservices. Since our business operation with the ARL is based on two entity which is users and books, using a monolith makes sense because there is no need for service discovery. This decreases the complexity of inter-component communication and makes deployment easy. The Docker Compose configuration provides orchestration over 3 components (MySQL, Spring Boot, Angular/Nginx) in one easily reproducible stack. Thus, the reviewer can run the entire environment with one command; this addresses the problem of different configurations on different machines.

3. FUNCTIONALITY AND USER EXPERIENCE

The ARL application offers a comprehensive system for managing books. User can create an account simply by entering a username, email address, password, and selecting an account type. The system enforces multiple levels of strict validation checks preventing duplicate username and email registrations.



The image shows a 'Create New Account' form. It has a title 'Create New Account' and a subtitle 'Fill in details to save your account to the database'. The form contains four input fields: 'Username' with the value 'Akbar', 'Email Address' with the value 'dthdfgdg@gmail.com', 'Password' with masked characters '....', and 'Account Type' with two radio button options: 'Student (View Only)' (selected) and 'Admin (Add & Delete Books)'. A blue 'Submit Account Data' button is at the bottom.

Create New Account

Fill in details to save your account to the database

Username:

Akbar

Email Address:

dthdfgdg@gmail.com

Password:

....

Account Type:

☒ Student (View Only)

☐ Admin (Add & Delete Books)

Submit Account Data

Duplicate username cannot be creating account

Create New Account

Fill in details to save your account to the database

Username is already taken.

Username:

tan_xuan_jie

Email Address:

dadadadadaad@gmail.com

Password:

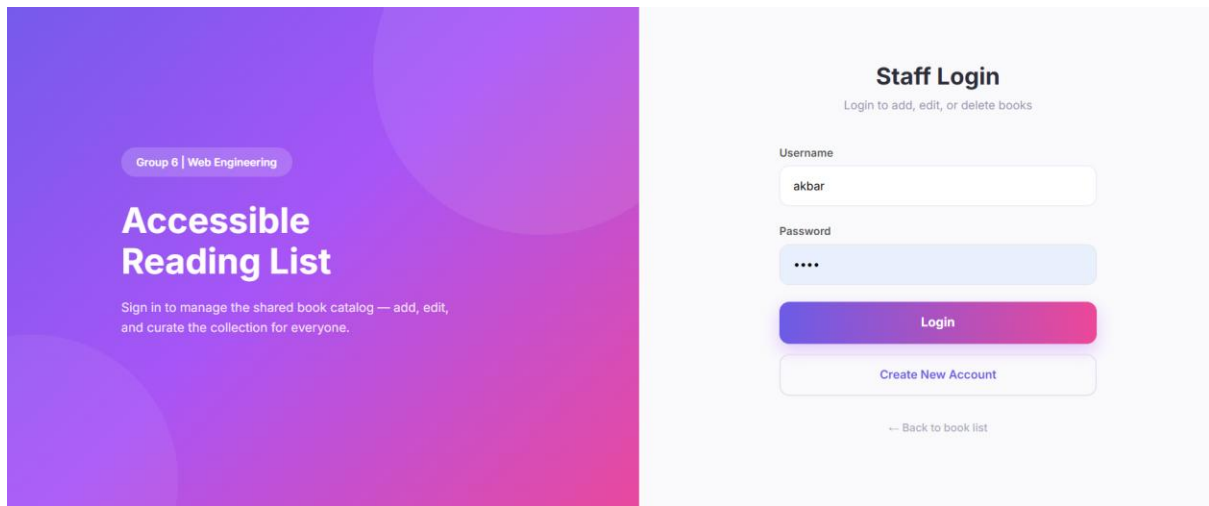
....

Account Type:

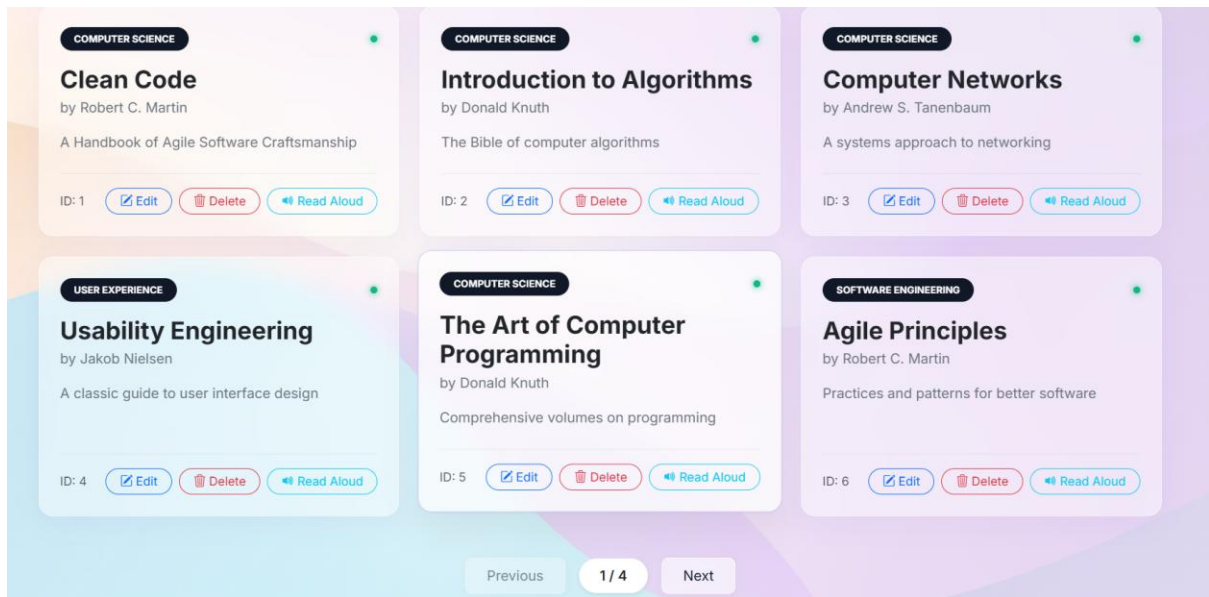
- ☒ Student (View Only)
☐ Admin (Add & Delete Books)

Submit Account Data

Users are directed to the Reading List Dashboard, after logging in to the ARL application. Dashboard access is based on user role. All student users may view and search or browse the list of books but only admin can add, edit and delete any book record.



The Reading List Dashboard displays books in a responsive card format that shows the book title, author, category, and description for each book. User can search the book they want by title, author or category. There is also a Text-to-Speech (TTS) function that will allow users who are visually impaired or auditory learners access to listen to a reading list managed by the ARL.



To create a new book entry with the Add New Book form, all fields are required such as title and author must have at least two characters, and description must have between 10 and 255 characters. Besides, all validation messages will only appear after the user interacts with the field like touched or modified field to reducing the likelihood of overwhelming users with an initial error state.

+ Add New Book

All fields marked with * are required

Title *

Example: Clean Code (minimum 2 characters)

Author *

Example: Robert C. Martin (minimum 2 characters)

Category *

Description *

Minimum 10 characters, maximum 255 characters

Prompt messages are displayed when the user fails to meet the requirements while entering data.

Title *

Enter book title

⚠ Title is required (minimum 2 characters)
Example: Clean Code (minimum 2 characters)

Author *

Enter author name

⚠ Author is required (minimum 2 characters)
Example: Robert C. Martin (minimum 2 characters)

Category *

-- Select a category --

⚠ Please select a category

Description *

Enter book description

⚠ Description is required (between 10 and 255 characters)
Minimum 10 characters, maximum 255 characters

Currently, these errors are displayed through alert () windows, this maybe disrupts the user experience. In the future, we will replace the existing backend error display method with non-blocking toast notifications to improve user experience.

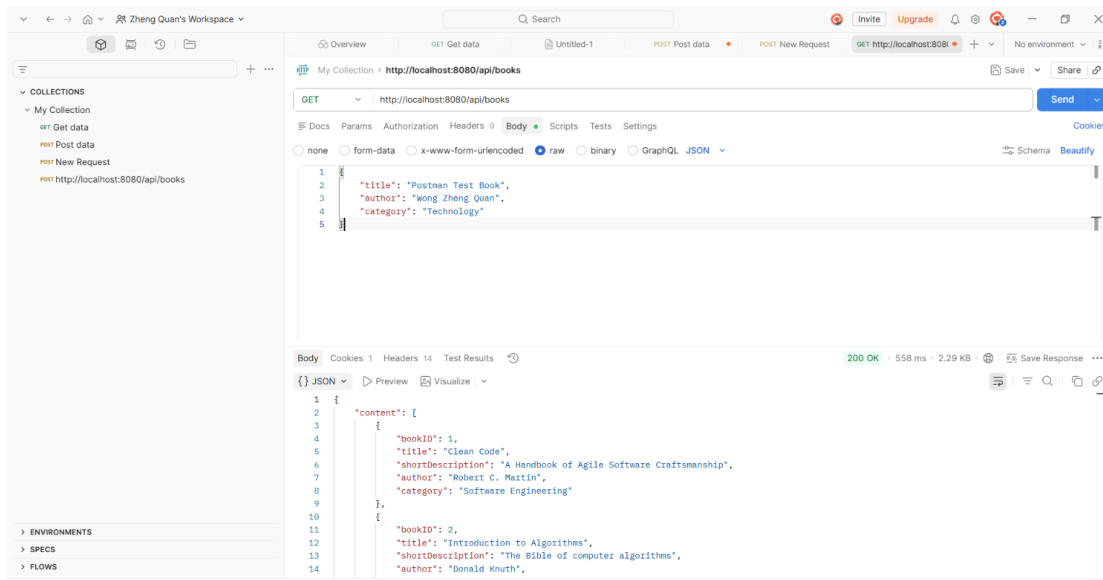
4. PERFORMANCE MEASUREMENT

We used Postman to conduct five consecutive requests on both core endpoints under local conditions. The results are summarised below:

API Endpoint	HTTP Method	Security Policy	Iterations	Average Time	Status Code
/api/books	GET	PermitAll	5 times	26.6ms	200 OK
/api/books	POST	PermitAll	5 times	32.2ms	201 Created

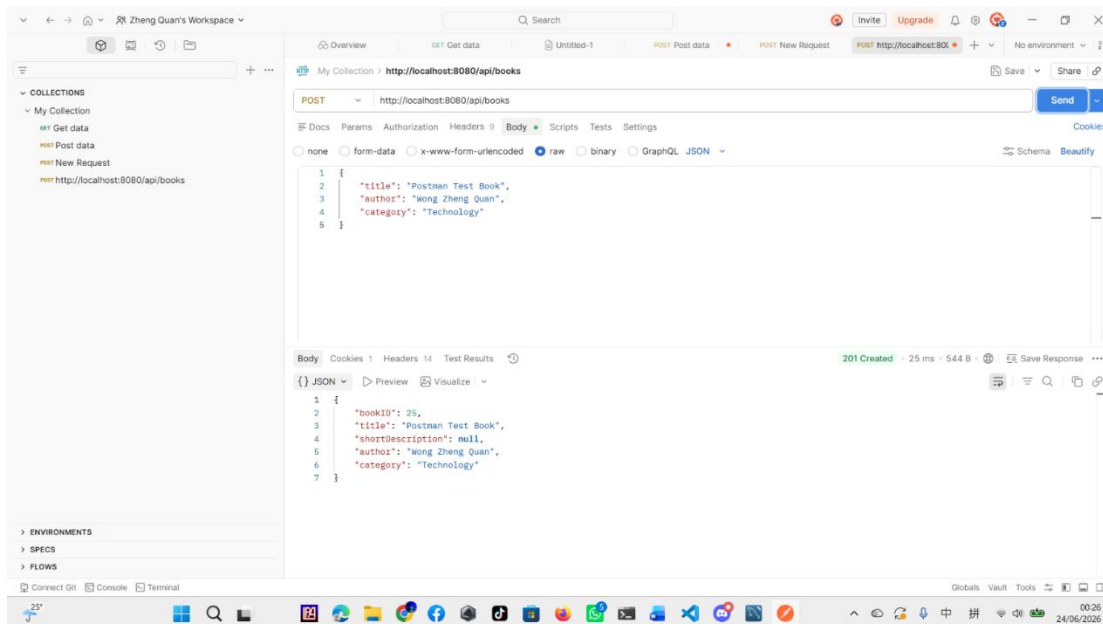
GET Method

For API Endpoint /api/books, the GET Method security policy is for permit all and get 5 times iterations return 200 OK with average time 26.6ms. Below picture show how we use Postman to do and result for this endpoint.



POST Method

For API Endpoint /api/books, the Post Method security policy is for permit all and get 5 times iterations return 201 Created with average time 32.2ms. Below picture show how we use Postman to do and result for this endpoint.

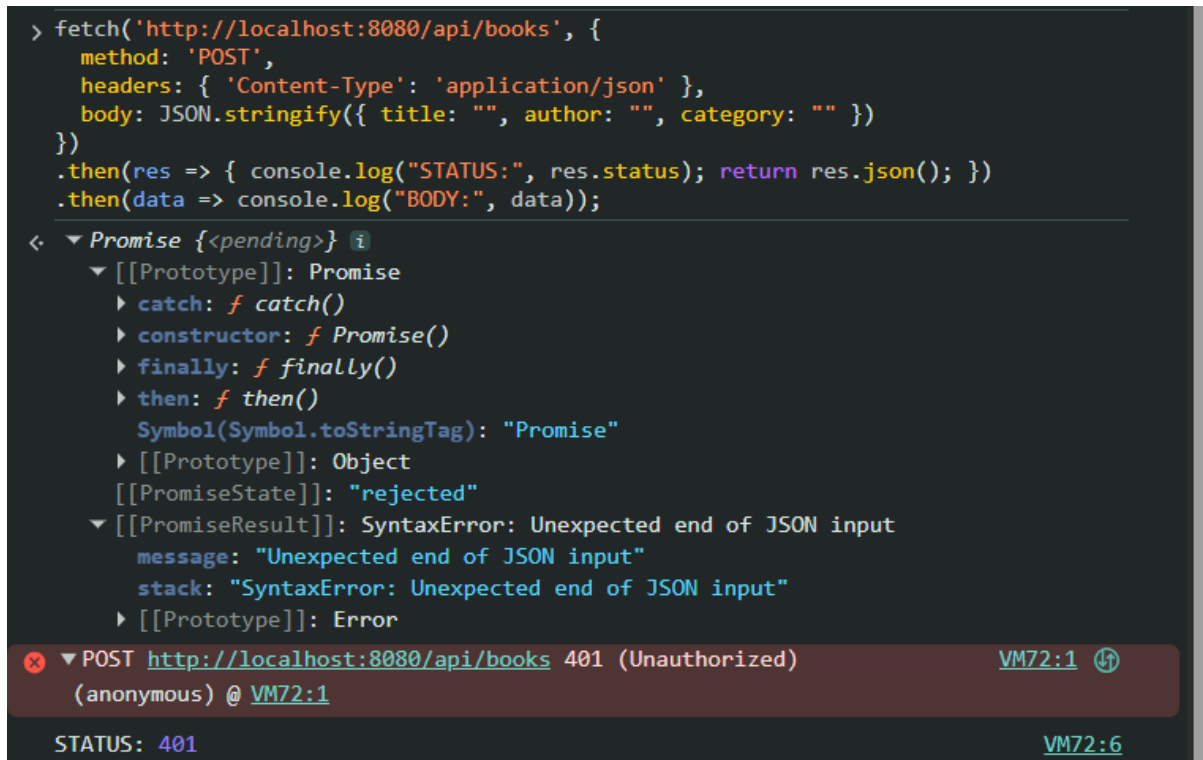


Both endpoints respond well within a 50 ms window, demonstrating efficient backend performance under local load. The difference between GET Method (26.6 ms) and POST Method (32.2 ms) is consistent with general software engineering expectations — write operations naturally carry more overhead than reads. Therefore, when we see the result ms of POST method is slightly higher than GET Method due to JSON deserialization, Bean Validation, and the additional I/O cost of a database write. Running `ng build --configuration` on the frontend can produced a minified bundle of approximately 350 KB, confirming the Angular build is optimized for fast initial page loads.

5. SECURITY RISK AND MITIGATION

5.1 Authentication and Authorization

SecurityConfig.java requires all write endpoints such as POST, PUT, and DELETE must mandate Basic Auth. However other endpoints like GETs are open to the public. Below picture show that anonymous users receive a 401 Unauthorized response when they try to perform a POST.



```
> fetch('http://localhost:8080/api/books', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ title: "", author: "", category: "" })
})
.then(res => { console.log("STATUS:", res.status); return res.json(); })
.then(data => console.log("BODY:", data));
```

▼ Promise {<pending>} i

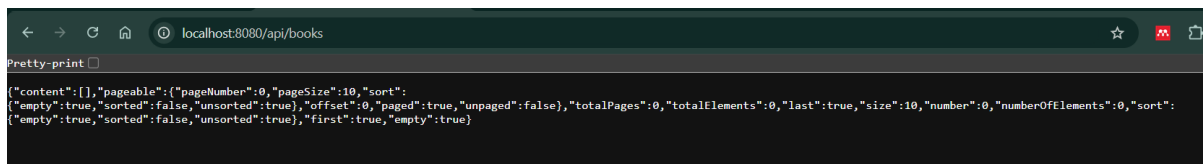
- [[Prototype]]: Promise
 - catch: f catch()
 - constructor: f Promise()
 - finally: f finally()
 - then: f then()
- Symbol(Symbol.toStringTag): "Promise"
- [[Prototype]]: Object
- [[PromiseState]]: "rejected"
- ▼ [[PromiseResult]]: SyntaxError: Unexpected end of JSON input
 - message: "Unexpected end of JSON input"
 - stack: "SyntaxError: Unexpected end of JSON input"
- [[Prototype]]: Error

✖ ▼ POST http://localhost:8080/api/books 401 (Unauthorized) VM72:1 ↕

(anonymous) @ VM72:1

STATUS: 401 VM72:6

Conversely, authenticated users sending a GET request will receive a 200 OK response along with a paginated list containing all books.



```
localhost:8080/api/books
```

Pretty-print

```
{
  "content": [],
  "pageable": {
    "pageNumber": 0,
    "pageSize": 10,
    "sort": {
      "empty": true,
      "sorted": false,
      "unsorted": true
    },
    "offset": 0,
    "paged": true,
    "unpaged": false,
    "totalPages": 0,
    "totalElements": 0,
    "last": true,
    "size": 10,
    "number": 0,
    "numberOfElements": 0,
    "sort": {
      "empty": true,
      "sorted": false,
      "unsorted": true
    },
    "first": true,
    "empty": true
  }
}
```

5.2 Input Validation and Error Handling

Book fields are validated on the server using Not Blank annotations. POST will be rejected by the server and return a structured 400 Bad Request response along with field level error messages through the GlobalExceptionHandler. If a user bypasses the Angular form and performs a POST with blank fields.

```
> fetch('http://localhost:8080/api/books', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': 'Basic ' + btoa('admin:password123')
  },
  body: JSON.stringify({ title: "", author: "", category: "" })
})
.then(res => { console.log("STATUS:", res.status); return res.json(); })
.then(data => console.log("BODY:", data));

< Promise {<pending>} i
  ▸ [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
    [[PromiseResult]]: undefined

✖ POST http://localhost:8080/api/books 400 (Bad Request) VM110:1
  (anonymous) @ VM110:1

STATUS: 400 VM110:9
BODY: VM110:10
  {author: 'Author is required', category: 'Category is required', title: 'Title is required'}
```

If user request non-existent book, the system will return 404 Not Found without exposing stack traces

```
> fetch('http://localhost:8080/api/books/9999')
.then(res => { console.log("STATUS:", res.status); return res.text(); })
.then(data => console.log("BODY:", data));

< Promise {<pending>} i
  ▸ [[Prototype]]: Promise
    [[PromiseState]]: "fulfilled"
    [[PromiseResult]]: undefined

✖ GET http://localhost:8080/api/books/9999 404 (Not Found) VM114:1
  STATUS: 404 VM114:2
  BODY: VM114:3
```

5.3 Risk OF Basic Auth and HTTP

Basic Auth credentials are Base64 encoded and therefore not encrypted. When the network uses plain HTTP, it is readable and susceptible to interception. Therefore, application must utilize TLS/SSL like Strict-Transport-Security to ensure security and prevent clickjacking and sniffing attacks.

6. TESTING

6.1 Backend JUnit Tests — Before Cloud Integration

Before cloud integration, Backend JUnit tests were performed in a local development environment that had XAMPP MySQL installed. This was accomplished by executing the command. `/mvnw test`, which triggered the Maven Surefire plugin to scan for and run all Java classes that ended in "Test."

```
PS C:\Users\sohai\OneDrive\文档\assignment web3\STIWK2124-A3-Group6\backend> ./mvnw test
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:demo >-----
[INFO] Building demo 0.0.1-SNAPSHOT
[INFO]    from pom.xml
[INFO] -----[ jar ]-----
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-surefire-plugin/3.1.2/maven-surefire-plugin-3.1.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/plugins/maven-surefire-plugin/3.1.2/maven-surefire-plugin-3.1.2.pom (5.5 kB at 15 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire/3.1.2/surefire-3.1.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire/3.1.2/surefire-3.1.2.pom (22 kB at 164 kB/s)
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ demo ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.11.0:compile (default-compile) @ demo ---
[INFO] Changes detected - recompiling the module! :input tree
[INFO] Compiling 16 source files with javac [debug release 17] to target/classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ demo ---
[INFO] skip non existing resourceDirectory C:\Users\sohai\OneDrive\文档\assignment web3\STIWK2124-A3-Group6\backend\src\test\resources
[INFO]
[INFO] --- compiler:3.11.0:testCompile (default-testCompile) @ demo ---
[INFO] No sources to compile
[INFO]
[INFO] --- surefire:3.1.2:test (default-test) @ demo ---
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/maven-surefire-common/3.1.2/maven-surefire-common-3.1.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/maven-surefire-common/3.1.2/maven-surefire-common-3.1.2.pom (6.1 kB at 79 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-api/3.1.2/surefire-api-3.1.2.pom
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-api/3.1.2/surefire-api-3.1.2.pom (3.5 kB at 62 kB/s)
```

```

surefire-booter-3.1.2.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-extensions-spi/3.1.2/surefire-extensions-spi-3.1.2.jar
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-extensions-api/3.1.2/surefire-extensions-api-3.1.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-logger-api/3.1.2/surefire-logger-api-3.1.2.jar (14 kB at 223 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/eclipse/aether/aether-util/1.0.0.v20140518/aether-util-1.0.0.v20140518.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/eclipse/aether/aether-util/1.0.0.v20140518/aether-util-1.0.0.v20140518.jar (146 kB at 510 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/eclipse/aether/aether-api/1.0.0.v20140518/aether-api-1.0.0.v20140518.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-extensions-api/3.1.2/surefire-extensions-api-3.1.2.jar (26 kB at 90 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-common-artifact-filters/3.1.1/maven-common-artifact-filters-3.1.1.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-extensions-spi/3.1.2/surefire-extensions-spi-3.1.2.jar (8.2 kB at 25 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/commons-io/commons-io/2.12.0/commons-io-2.12.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-booter/3.1.2/surefire-booter-3.1.2.jar (118 kB at 350 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-shared-utils/3.1.2/surefire-shared-utils-3.1.2.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/shared/maven-common-artifact-filters/3.1.1/maven-common-artifact-filters-3.1.1.jar (61 kB at 181 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-api/3.1.2/surefire-api-3.1.2.jar (171 kB at 453 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/eclipse/aether/aether-api/1.0.0.v20140518/aether-api-1.0.0.v20140518.jar (136 kB at 356 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/commons-io/commons-io/2.12.0/commons-io-2.12.0.jar (474 kB at 727 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-shared-utils/3.1.2/surefire-shared-utils-3.1.2.jar (2.3 MB at 3.0 MB/s)
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 6.891 s
[INFO] Finished at: 2026-06-23T21:59:18+08:00
[INFO] -----

```

The test result is 3 core business and security test cases ran with 0 failures and errors. This means that all assertions matched successfully. The final console output successfully threw a BUILD SUCCESS, thereby indicating the successful execution of the tests and correctness of all security interception logic for the repositories, services, and controller layers of the backend.

6.2 Backend JUnit Tests — After Cloud Integration (CI)

After cloud integration occurred, all backend JUnit tests were executed again through the CI pipeline of GitHub Actions, which automatically executes the pipeline each time there is a push to the main branch of the Git repository.

6.3 Frontend Angular Unit Tests

The AuthService unit test uses Angulara HttpClientTestingModule to test the frontend. In this test has four tests, which is check services created,for the function isLoggedIn() returns false, if using an undefined token, isLoggedIn() returns true, if using a valid token, local storage is cleared when logging out. In this project, all four test cases passed successfully, this proves that the authentication state is managing correctly the client.

```
PS C:\Users\User\Documents\web engineering\assign3\STIWK2124-A3-Group6\frontend> npm run test

| Initial total | 11.88 kB

Application bundle generation complete. [1.337 seconds] - 2026-06-23T18:16:14.837Z

Watch mode enabled. Watching for file changes...

DEV v4.1.7 C:/Users/User/Documents/web engineering/assign3/STIWK2124-A3-Group6/frontend

✓ STIWK2124-A2-Group6 src/app/services/auth.service.spec.ts (4 tests) 15ms
```

7. NEXT STEPS

These technologies can be integrated into the system in the future to enhance overall security and quality.

1. Enhancements for Security and Authentication

- **Using JSON Web Token (JWT):** Now system is use basic authentication, when we switch to JSON Web Token (JWT) based approaches can allow exchanging credentials between parties only during the login process. JWT features such as token expiration and token signature verification can contribute to reducing the risk of disclosing credentials.
- **TLS and Reverse Proxies:** Using NGINX can allow for the enforcing of HTTPS communication. This can allow systems to use HTTP headers to protect against interception of communication and common web-based attacks.
- **Implement Rate Limiting:** Using tools like Bucket4J can help prevent brute force login attempts. This is because it can implement rate limiting on available requests based on IP address.

2. Improvements to User Experience

- **Modern Notification System:** Alert dialogs presented from the browser can be replaced with angular material or ngx-toastr based toast notifications. This will be providing for less obtrusive and more seamless feedback mechanisms for users.
- **Pagination Controls for the Frontend:** Although pagination is implemented at the backend of the system. However, adding visible pagination controls at the Angular interface can easier browsing and enable better performance of the book catalog as it continues to grow.

3. Quality Assurance & Testing

- **End-to-End (E2E) Test Automation:** Using Cypress or Playwright for E2E test of important user workflow such as registration, book management and else in future developments. Therefore

8. CONCLUSION

In conclusion, the ARL application is compliant with all requirements for Assignment 3. JUnit tests for the backend cover functional, boundary and security / defense scenarios. All the tests for the backend have passed and show the BUILD SUCCESS both locally and in the GitHub actions Continuous Integration (CI) pipeline.

The client-side Angular unit tests confirm 4 separate cases of successful authentication for the application with the use of Basic Authentication at the API layer with role-based access, server-side Bean Validation and structured 400/404 error responses to clients. Evidence of the above-mentioned security enforcement can be viewed in the form of screenshots provided with this report. Performance benchmarks indicate both API core endpoints are returning within 35 ms and that an optimized Nginx config bundle will result from the use of the Angular production build process.

The use of Docker Compose to deploy a reproducible 1-command deployable stack will ensure the application can be migrated to production. The design decisions made (Basic Authentication, Monolith, Standalone Angular Component) have been made pragmatically and in accordance with the scope of the assignment, while providing a roadmap to document in the next steps above for further development towards a hardened production release of this application.